Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

Домашняя работа №2

«Сравнение скорости выполнения вычислений в браузере с помощью JS и WASM»

по дисциплине: «Фреймворки web-приложений»

Выполнил: студент группы ИУК4-82Б _____ (Подпись) Моряков В.Ю. (И.О. Фамилия)

Проверил: _____ (Подпись) Гришунов С.С. (И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026

Задание
Реализовать 2 решения задачи в браузере по варинату: на JS и WASM. Сгенерировать случайные наборы входных данных большого размера (от 10тыс.) и сравнить время выполнения двух решений.

Требования к реализации

- Можно использовать любой ЯП, компилируемый в WASM (предпочтительно Rust)
- Можно использовать любые JS-библиотеки и фреймворки для отображения данных
- (но не для выполнения вычислений!)

Вариант: Поиск кратчайшего пути в графе дорог.

Страницы:

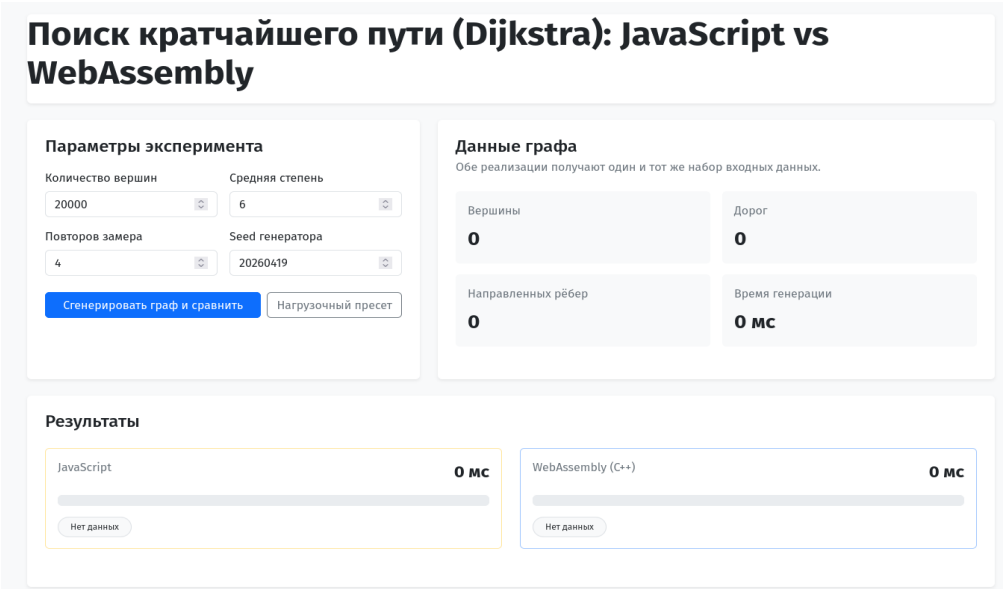

Рисунок 1 Главная страница

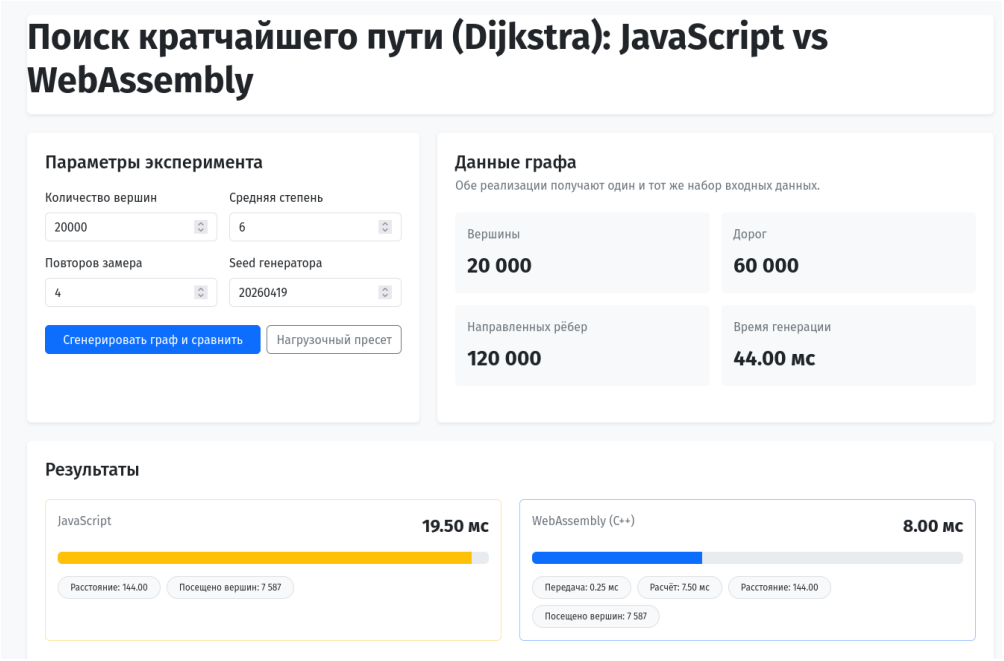

Рисунок 2 Результаты работы программы

Листинг:

```
================================================================================

cpp-wasm/Graph.cpp

#include "Graph.h"

Graph::Graph(std::uint32_t node_count, std::uint32_t edge_count,
             const std::uint32_t *from, const std::uint32_t *to,
             const double *weights)
    : node_count_(node_count), adjacency_(node_count) {

  for (std::uint32_t edge_index = 0; edge_index < edge_count; edge_index += 1) {
    const std::uint32_t source = from[edge_index];
    const std::uint32_t destination = to[edge_index];
    if (source >= node_count_ || destination >= node_count_) {
      is_valid_ = false;
      adjacency_.clear();
      return;
    }

    adjacency_[source].push_back({destination, weights[edge_index]});
  }
}

std::uint32_t Graph::node_count() const { return node_count_; }

bool Graph::is_valid() const { return is_valid_; }

const std::vector<Graph::edge> &Graph::neighbors(std::uint32_t node) const {
  return adjacency_[node];
}

================================================================================

cpp-wasm/Graph.h

#ifndef __GRAPH_H__
#define __GRAPH_H__

#include <cstdint>
#include <vector>

class Graph {
public:
  struct edge {
    std::uint32_t to;
    double weight;
  };

  Graph(std::uint32_t node_count, std::uint32_t edge_count,
        const std::uint32_t *from, const std::uint32_t *to,
        const double *weights);

  std::uint32_t node_count() const;
  bool is_valid() const;
  const std::vector<edge> &neighbors(std::uint32_t node) const;

private:
```

```
  std::uint32_t node_count_;
  bool is_valid_ = true;
  std::vector<std::vector<edge>> adjacency_;
};

#endif
```

==========

cpp-wasm/Path_Finder.cpp

```
#include "Graph.h"
#include "Shortest_Path_Finder.h"

#include <cstdint>

namespace {
constexpr std::uint32_t max_node_count = 100000;
constexpr std::uint32_t max_directed_edges = 2000000;
} // namespace

extern "C" {
int run_shortest_path(std::uint32_t node_count, std::uint32_t edge_count,
                      const std::uint32_t *from, const std::uint32_t *to,
                      const double *weights, std::uint32_t start,
                      std::uint32_t target, std::uint32_t *found,
                      double *distance, std::uint32_t *visited_count) {
  if (node_count == 0 || node_count > max_node_count) {
    return 1;
  }
  if (edge_count > max_directed_edges) {
    return 2;
  }
  if (start >= node_count || target >= node_count) {
    return 3;
  }

  const Graph graph_instance(node_count, edge_count, from, to, weights);
  const Shortest_Path_Finder finder;
  Shortest_Path_Finder::result result;
  const int error_code = finder.run(graph_instance, start, target, result);
  if (error_code != 0) {
    return error_code;
  }

  *found = result.found ? 1U : 0U;
  *distance = result.found ? result.distance : 0.0;
  *visited_count = result.visited_count;

  return 0;
}
}
```

==========

cpp-wasm/Shortest_Path_Finder.cpp

```
#include <limits>
#include <queue>
#include <utility>
#include <vector>
```

```
#include "Shortest_Path_Finder.h"

namespace {
constexpr double inf = std::numeric_limits<double>::infinity();
}

int Shortest_Path_Finder::run(const Graph &graph, std::uint32_t start,
                              std::uint32_t target, result &result) const {
  if (!graph.is_valid()) {
    return 4;
  }

  using queue_entry = std::pair<double, std::uint32_t>;

  std::vector<double> distances(graph.node_count(), inf);
  std::vector<bool> visited(graph.node_count(), false);
  std::priority_queue<queue_entry, std::vector<queue_entry>, std::greater<>>
      queue;

  result = {};
  distances[start] = 0.0;
  queue.push({0.0, start});

  while (!queue.empty()) {
    const auto [current_cost, node] = queue.top();
    queue.pop();

    if (visited[node]) {
      continue;
    }

    visited[node] = true;
    result.visited_count += 1;

    if (node == target) {
      break;
    }

    if (current_cost > distances[node]) {
      continue;
    }

    for (const auto &[next_node, weight] : graph.neighbors(node)) {
      const double next_cost = current_cost + weight;
      if (next_cost >= distances[next_node]) {
        continue;
      }

      distances[next_node] = next_cost;
      queue.push({next_cost, next_node});
    }
  }

  if (distances[target] == inf) {
    return 0;
  }

  result.found = true;
  result.distance = distances[target];
  return 0;
}
```

==========

cpp-wasm/Shortest_Path_Finder.h

```
#ifndef __SHORTES_PATH_FINDNER_H__
#define __SHORTES_PATH_FINDNER_H__

#include <cstdint>

#include "Graph.h"

class Shortest_Path_Finder {
public:
  struct result {
    bool found = false;
    double distance = 0.0;
    std::uint32_t visited_count = 0;
  };

  int run(const Graph &graph, std::uint32_t start, std::uint32_t target,
          result &result) const;
};

#endif
```

==========

cpp-wasm/wasm_runtime.cpp

```
#include <cstdlib>
#include <vector>

#include "wasm_runtime.h"

namespace {
std::vector<void *> wasm_allocations;
}

extern "C" {
std::uint32_t alloc_bytes(std::uint32_t byte_count) {
  void *pointer = std::malloc(byte_count == 0 ? 1 : byte_count);
  if (pointer == nullptr) {
    return 0;
  }

  wasm_allocations.push_back(pointer);
  return static_cast<std::uint32_t>(reinterpret_cast<std::uintptr_t>(pointer));
}

void reset_allocator() {
  for (void *pointer : wasm_allocations) {
    std::free(pointer);
  }
  wasm_allocations.clear();
}
}
```

==========

cpp-wasm/wasm_runtime.h

```
#ifndef __WASM_RUNTIME_H__
```

```
#define __WASM_RUNTIME_H__

#include <cstdint>

extern "C" {
std::uint32_t alloc_bytes(std::uint32_t byte_count);
void reset_allocator();
}

#endif

================================================================================

src/Graph.js

export class Graph {
  constructor(node_count) {
    this.node_count = node_count;
    this.from = [];
    this.to = [];
    this.weights = [];
    this.edge_ids = new Set();
  }

  add_edge(a, b, weight) {
    if (a === b) {
      return false;
    }

    const min_node = Math.min(a, b);
    const max_node = Math.max(a, b);
    const id = min_node * this.node_count + max_node;

    if (this.edge_ids.has(id)) {
      return false;
    }

    this.edge_ids.add(id);
    this.from.push(a, b);
    this.to.push(b, a);
    this.weights.push(weight, weight);
    return true;
  }

  build_compressed_sparse_row() {
    if (this.from.length !== this.to.length || this.to.length !== this.weights.length)
{
      throw new Error("Массивы рёбер должны иметь одинаковую длину.");
    }

    const row_ptr = new Uint32Array(this.node_count + 1);
    for (let index = 0; index < this.from.length; index += 1) {
      row_ptr[this.from[index] + 1] += 1;
    }

    for (let index = 1; index < row_ptr.length; index += 1) {
      row_ptr[index] += row_ptr[index - 1];
    }

    const cursor = row_ptr.slice(0, this.node_count);
    const neighbors = new Uint32Array(this.to.length);
    const costs = new Float64Array(this.weights.length);
```

```
    for (let index = 0; index < this.from.length; index += 1) {
      const source = this.from[index];
      const slot = cursor[source];
      neighbors[slot] = this.to[index];
      costs[slot] = this.weights[index];
      cursor[source] += 1;
    }

    return { offsets: row_ptr, neighbors, costs };
  }

  to_dataset({ seed, start, target }) {
    const from = Uint32Array.from(this.from);
    const to = Uint32Array.from(this.to);
    const weights = Float64Array.from(this.weights);

    return {
      node_count: this.node_count,
      seed,
      start,
      target,
      undirected_edge_count: this.edge_ids.size,
      directed_edge_count: this.from.length,
      from,
      to,
      weights,
      csr: this.build_compressed_sparse_row()
    };
  }
}

export function build_compressed_sparse_row(node_count, from, to, weights) {
  const graph = new Graph(node_count);
  graph.from = Array.from(from);
  graph.to = Array.from(to);
  graph.weights = Array.from(weights);
  return graph.build_compressed_sparse_row();
}

================================================================================

src/Graph_Generator.js

import { Graph } from "./Graph.js";

export class Graph_Generator {
  constructor(seed) {
    this.rng = Graph_Generator.create_mulberry_32(seed);
  }

  static create_mulberry_32(seed) {
    let state = seed >>> 0;

    // https://github.com/cprosche/mulberry32
    return () => {
      state += 0x6d2b79f5;
      let value = state;
      value = Math.imul(value ^ (value >>> 15), value | 1);
      value ^= value + Math.imul(value ^ (value >>> 7), value | 61);
      return ((value ^ (value >>> 14)) >>> 0) / 4294967296;
    };
```

```
  }

  random_int(max_ex) {
    return Math.floor(this.rng() * max_ex);
  }

  random_weight() {
    return 1 + Math.floor(this.rng() * 90);
  }

  generate_road_graph({ node_count, average_degree, seed }) {
    const normalized_degree = Math.max(2, Math.min(average_degree, node_count - 1));
    const target_undirected_edges = Math.max(node_count - 1, Math.floor((node_count *
normalized_degree) / 2));
    const graph = new Graph(node_count);

    // гарантируем что граф будет связным
    for (let node = 1; node < node_count; node++) {
      graph.add_edge(node - 1, node, this.random_weight());
    }

    while (graph.edge_ids.size < target_undirected_edges) {
      const a = this.random_int(node_count);
      let b = this.random_int(node_count - 1);
      if (b >= a) {
        b += 1;
      }

      graph.add_edge(a, b, this.random_weight());
    }

    const start = this.random_int(node_count);
    let target = this.random_int(node_count - 1);
    if (target >= start) {
      target += 1;
    }

    return graph.to_dataset({ seed, start, target });
  }
}

export function generate_road_graph(config) {
  const generator = new Graph_Generator(config.seed);
  return generator.generate_road_graph(config);
}

================================================================================

src/Shortest_Path_Finder.js

import { build_compressed_sparse_row } from "./Graph.js";

class Emu_Priority_Queue {
  constructor() {
    this.items = [];
  }

  push(node, cost) {
    this.items.push({ node, cost });
    this.#bubble_up(this.items.length - 1);
  }
```

```
  pop() {
    if (this.items.length === 0) {
      return null;
    }

    const top = this.items[0];
    const last = this.items.pop();

    if (this.items.length > 0 && last) {
      this.items[0] = last;
      this.#bubble_down(0);
    }

    return top;
  }

  get size() {
    return this.items.length;
  }

  #bubble_up(index) {
    let current_index = index;
    while (current_index > 0) {
      const parent_index = Math.floor((current_index - 1) / 2);
      if (this.#compare(this.items[parent_index], this.items[current_index]) <= 0) {
        break;
      }

      [this.items[parent_index], this.items[current_index]] =
[this.items[current_index], this.items[parent_index]];
      current_index = parent_index;
    }
  }

  #bubble_down(index) {
    let current_index = index;

    while (true) {
      const left_child = current_index * 2 + 1;
      const right_child = left_child + 1;
      let smallest = current_index;

      if (left_child < this.items.length && this.#compare(this.items[left_child],
this.items[smallest]) < 0) {
        smallest = left_child;
      }

      if (right_child < this.items.length && this.#compare(this.items[right_child],
this.items[smallest]) < 0) {
        smallest = right_child;
      }

      if (smallest === current_index) {
        break;
      }

      [this.items[current_index], this.items[smallest]] = [this.items[smallest],
this.items[current_index]];
      current_index = smallest;
    }
  }
```

```
  #compare(left, right) {
    if (left.cost !== right.cost) {
      return left.cost - right.cost;
    }

    return left.node - right.node;
  }
}

export class Shortest_Path_Finder {
  find_shortest_path(graph, start, target, prebuilt_csr) {
    if (start < 0 || start >= graph.node_count || target < 0 || target >=
graph.node_count) {
      throw new Error("Некорректные вершины старта или финиша.");
    }

    const build_started = performance.now();
    const { offsets: row_ptrs, neighbors, costs } =
      prebuilt_csr ?? build_compressed_sparse_row(graph.node_count, graph.from,
graph.to, graph.weights);
    const build_time = performance.now() - build_started;

    const distances = new Float64Array(graph.node_count);
    distances.fill(Number.POSITIVE_INFINITY);

    const visited = new Uint8Array(graph.node_count);
    const heap = new Emu_Priority_Queue();
    let visited_count = 0;

    const compute_started = performance.now();
    distances[start] = 0;
    heap.push(start, 0);

    while (heap.size > 0) {
      const state = heap.pop();
      if (!state) {
        break;
      }

      const { node, cost } = state;
      if (visited[node]) {
        continue;
      }

      visited[node] = 1;
      visited_count += 1;

      if (node === target) {
        break;
      }

      if (cost > distances[node]) {
        continue;
      }

      for (let edge_index = row_ptrs[node]; edge_index < row_ptrs[node + 1];
edge_index += 1) {
        const next_node = neighbors[edge_index];
        const next_cost = cost + costs[edge_index];

        if (next_cost < distances[next_node]) {
          distances[next_node] = next_cost;
```

```
          heap.push(next_node, next_cost);
        }
      }
    }

    const found = Number.isFinite(distances[target]);
    const compute_time = performance.now() - compute_started;
    return {
      found,
      distance: found ? distances[target] : null,
      visited_count,
      timings: {
        build: build_time,
        compute: compute_time
      }
    };
  }
}

export function shortest_path_js(node_count, from, to, weights, start, target,
prebuilt_csr) {
  const graph = { node_count, from, to, weights };
  const finder = new Shortest_Path_Finder();
  return finder.find_shortest_path(graph, start, target, prebuilt_csr);
}

================================================================================

src/main.js

import "bootstrap/dist/css/bootstrap.min.css";

import { generate_road_graph } from "./Graph_Generator.js";
import { shortest_path_js } from "./Shortest_Path_Finder.js";
import init_wasm, { shortest_path_wasm } from "./wasm/path_finder_wasm.js";

(() => {
  class UI_context {
    constructor() {
      this.form = document.querySelector("#benchmark-form");
      this.controls = {
        run_button: document.querySelector("#run-button"),
        preset_button: document.querySelector("#preset-button")
      };
      this.dataset_fields = {
        nodes: document.querySelector("#nodes-stat"),
        edges: document.querySelector("#edges-stat"),
        directed: document.querySelector("#directed-stat"),
        generation: document.querySelector("#generation-stat")
      };
      this.result_fields = {
        js_average: document.querySelector("#js-avg"),
        js_details: document.querySelector("#js-details"),
        js_bar: document.querySelector("#js-bar"),
        wasm_average: document.querySelector("#wasm-avg"),
        wasm_details: document.querySelector("#wasm-details"),
        wasm_bar: document.querySelector("#wasm-bar"),
        final_result: document.querySelector("#final_result")
      };
    }

    read_configuration() {
```

```
      const form_data = new FormData(this.form);
      const node_count = Number(form_data.get("node_count"));
      const average_degree = Number(form_data.get("average_degree"));
      const runs = Number(form_data.get("runs"));
      const seed = Number(form_data.get("seed"));

      return {
        node_count: Math.max(10000, Math.min(100000, Math.trunc(node_count))),
        average_degree: Math.max(2, Math.min(20, Math.trunc(average_degree))),
        runs: Math.max(1, Math.min(10, Math.trunc(runs))),
        seed: Math.max(1, Math.min(4294967295, Math.trunc(seed)))
      };
    }

    write_configuration(config) {
      this.form.elements.namedItem("node_count").value = String(config.node_count);
      this.form.elements.namedItem("average_degree").value =
String(config.average_degree);
      this.form.runs.value = String(config.runs);
      this.form.seed.value = String(config.seed);
    }

    reset_result_bars() {
      this.result_fields.js_bar.style.width = "0%";
      this.result_fields.wasm_bar.style.width = "0%";
    }

    render_dataset(graph, generation_time) {
      this.dataset_fields.nodes.textContent = format_integer(graph.node_count);
      this.dataset_fields.edges.textContent =
format_integer(graph.undirected_edge_count);
      this.dataset_fields.directed.textContent =
format_integer(graph.directed_edge_count);
      this.dataset_fields.generation.textContent =
format_milliseconds(generation_time);
    }

    set_running_state(is_running) {
      this.controls.run_button.disabled = is_running;
      this.controls.preset_button.disabled = is_running;
      this.controls.run_button.classList.toggle("disabled", is_running);
      this.controls.preset_button.classList.toggle("disabled", is_running);
    }
  }

  const ui_context = new UI_context();

  let is_wasm_ready = false;

  const format_integer = (value) => {
    return new Intl.NumberFormat("ru-RU").format(value);
  }

  const format_milliseconds = (value) => {
    return `${value.toFixed(2)} мс`;
  }

  const format_distance = (value) => {
    if (value === null || !Number.isFinite(value)) {
      return "нет пути";
    }
```

```
    return value.toFixed(2);
  }

  const render_detail_items = (container, items) => {
    container.replaceChildren();

    for (const item of items) {
      const element = document.createElement("span");
      element.className = "badge rounded-pill text-bg-light border text-secondary-
emphasis fw-normal py-2 px-3";
      element.textContent = item;
      container.appendChild(element);
    }
  }

  const calculate_average = (values) => {
    return values.reduce((sum, value) => sum + value, 0) / values.length;
  }

  function approximately_equal(left, right) {
    if (left === right) {
      return true;
    }

    if (left === null || right === null) {
      return false;
    }

    return Math.abs(left - right) < 1e-9;
  }

  function validate_results(js_result, wasm_result) {
    if (js_result.found !== wasm_result.found) {
      throw new Error("JS и WASM вернули разные признаки существования пути.");
    }

    if (!approximately_equal(js_result.distance, wasm_result.distance)) {
      throw new Error("JS и WASM вычислили разные расстояния.");
    }

    if (js_result.visited_count !== wasm_result.visited_count) {
      throw new Error("JS и WASM посетили разное количество вершин.");
    }
  }

  function render_results(js_runs, wasm_runs, js_result, wasm_result) {
    const js_average = calculate_average(js_runs);
    const wasm_average = calculate_average(wasm_runs);
    const max_value = Math.max(js_average, wasm_average, 1);

    ui_context.result_fields.js_average.textContent = format_milliseconds(js_average);
    ui_context.result_fields.wasm_average.textContent =
format_milliseconds(wasm_average);
    render_detail_items(ui_context.result_fields.js_details, [
      `Расстояние: ${format_distance(js_result.distance)}`,
      `Посещено вершин: ${format_integer(js_result.visited_count)}`
    ]);
    render_detail_items(ui_context.result_fields.wasm_details, [
      `Передача: ${format_milliseconds(wasm_result.timings.transfer)}`,
      `Расчёт: ${format_milliseconds(wasm_result.timings.compute)}`,
      `Расстояние: ${format_distance(wasm_result.distance)}`,
      `Посещено вершин: ${format_integer(wasm_result.visited_count)}`
```

```
    ]);
    ui_context.result_fields.js_bar.style.width = `${(js_average / max_value) *
100}%`;
    ui_context.result_fields.wasm_bar.style.width = `${(wasm_average / max_value) *
100}%`;
    // ui_context.result_fields.final_result = `${js_average}%`
  }

  async function benchmark_implementation(label, runner, graph) {
    const start_time = performance.now();
    const result = await runner(graph);
    const elapsed = performance.now() - start_time;
    console.log(`${label}: ${format_milliseconds(elapsed)}`);
    return { elapsed, result };
  }

  const run_benchmark = async (event) => {
    event.preventDefault();

    if (!is_wasm_ready) {
      console.error("WASM IS NOT LOADED")
      return;
    }

    const config = ui_context.read_configuration();
    ui_context.set_running_state(true);
    ui_context.reset_result_bars();

    try {
      const generation_started = performance.now();
      const graph = generate_road_graph(config);
      const generation_time = performance.now() - generation_started;

      console.info("GENERATION TIME: ", generation_time)

      ui_context.render_dataset(graph, generation_time);

      console.log(`Сгенерирован граф: ${format_integer(graph.node_count)} вершин, $
{format_integer(graph.undirected_edge_count)} дорог.`);
      console.log(`Маршрут: ${graph.start} -> ${graph.target}.`);

      // shortest_path_js(graph.node_count, graph.from, graph.to, graph.weights,
graph.start, graph.target);
      // shortest_path_wasm(graph.node_count, graph.from, graph.to, graph.weights,
graph.start, graph.target);

      const js_runs = [];
      const wasm_runs = [];
      const js_timing_runs = [];
      const wasm_timing_runs = [];
      let js_result = null;
      let wasm_result = null;

      for (let run_index = 0; run_index < config.runs; run_index++) {
        const js_runner = (current_graph) => shortest_path_js(
          current_graph.node_count,
          current_graph.from,
          current_graph.to,
          current_graph.weights,
          current_graph.start,
          current_graph.target
        );
```

```
  const wasm_runner = (current_graph) => shortest_path_wasm(
    current_graph.node_count,
    current_graph.from,
    current_graph.to,
    current_graph.weights,
    current_graph.start,
    current_graph.target
  );

  let first_name;
  let first_runner;
  let second_name;
  let second_runner;

  if (run_index % 2 === 0) {
    first_name = "JS";
    first_runner = js_runner;

    second_name = "WASM";
    second_runner = wasm_runner;
  } else {
    first_name = "WASM";
    first_runner = wasm_runner;

    second_name = "JS";
    second_runner = js_runner;
  }

  const first_measurement = await benchmark_implementation(
    first_name,
    first_runner,
    graph
  );

  const second_measurement = await benchmark_implementation(
    second_name,
    second_runner,
    graph
  );

  if (first_name === "JS") {
    js_runs.push(first_measurement.elapsed);
    js_result = first_measurement.result;
    js_timing_runs.push(first_measurement.result.timings);

    wasm_runs.push(second_measurement.elapsed);
    wasm_result = second_measurement.result;
    wasm_timing_runs.push(second_measurement.result.timings);
  } else {
    js_runs.push(second_measurement.elapsed);
    js_result = second_measurement.result;
    js_timing_runs.push(second_measurement.result.timings);

    wasm_runs.push(first_measurement.elapsed);
    wasm_result = first_measurement.result;
    wasm_timing_runs.push(first_measurement.result.timings);
  }

  validate_results(js_result, wasm_result);
}
```

```
      js_result.timings = {
        build: calculate_average(js_timing_runs.map((timing) => timing.build)),
        compute: calculate_average(js_timing_runs.map((timing) => timing.compute))
      };
      wasm_result.timings = {
        transfer: calculate_average(wasm_timing_runs.map((timing) =>
timing.transfer)),
        compute: calculate_average(wasm_timing_runs.map((timing) => timing.compute))
      };

      render_results(js_runs, wasm_runs, js_result, wasm_result);
    } catch (error) {
      console.error(error);
    } finally {
      ui_context.set_running_state(false);
    }
  }

  const initialize_wasm = async () => {
    try {
      await init_wasm();
      is_wasm_ready = true;
      console.log("WASM-модуль загружен.");
    } catch (error) {
      console.error(error);
    }
  }

  ui_context.form.addEventListener("submit", run_benchmark);

  ui_context.controls.preset_button.addEventListener("click", () => {
    ui_context.write_configuration({
      node_count: 50000,
      average_degree: 8,
      runs: 3,
      seed: 20260501
    });
  });

  initialize_wasm();
})()

================================================================================

src/wasm/path_finder_wasm.js

let wasm_exports = null;
let wasm_initialization = null;

function create_wasi_shim() {
  return {
    wasi_snapshot_preview1: {
      args_get() {
        return 0;
      },
      args_sizes_get() {
        return 0;
      },
      fd_close() {
        return 0;
      },
      fd_seek() {
```

```
        return 0;
      },
      fd_write() {
        return 0;
      },
      proc_exit(code) {
        throw new Error(`WASM вызвал proc_exit(${code}).`);
      }
    }
  };
}

function get_memory() {
  if (!wasm_exports) {
    throw new Error("WASM-модуль ещё не инициализирован.");
  }

  return wasm_exports.memory;
}

async function normalize_input(input) {
  if (input instanceof URL || typeof input === "string") {
    const response = await fetch(input);
    return response.arrayBuffer();
  }

  if (input instanceof Response) {
    return input.arrayBuffer();
  }

  if (input instanceof ArrayBuffer) {
    return input;
  }

  if (ArrayBuffer.isView(input)) {
    return input.buffer.slice(input.byteOffset, input.byteOffset + input.byteLength);
  }

  throw new Error("Неподдерживаемый формат входа для инициализации WASM.");
}

function copy_uint32_array(values) {
  const pointer = wasm_exports.alloc_bytes(values.byteLength);
  new Uint32Array(get_memory().buffer, pointer, values.length).set(values);
  return pointer;
}

function copy_float64_array(values) {
  const pointer = wasm_exports.alloc_bytes(values.byteLength);
  new Float64Array(get_memory().buffer, pointer, values.length).set(values);
  return pointer;
}

function alloc_uint32(count = 1) {
  return wasm_exports.alloc_bytes(count * Uint32Array.BYTES_PER_ELEMENT);
}

function alloc_float64(count = 1) {
  return wasm_exports.alloc_bytes(count * Float64Array.BYTES_PER_ELEMENT);
}

function map_error_code(error_code) {
```

```
  switch (error_code) {
    case 1:
      return "Количество вершин превышает допустимый диапазон.";
    case 2:
      return "Количество рёбер превышает допустимый диапазон.";
    case 3:
      return "Некорректные вершины старта или финиша.";
    case 4:
      return "В массивах рёбер есть вершина вне диапазона.";
    default:
      return `WASM вернул неизвестный код ошибки: ${error_code}.`;
  }
}

export default async function init_wasm(input = new URL("./path_finder_wasm.wasm",
import.meta.url)) {
  if (wasm_exports) {
    return wasm_exports;
  }

  if (!wasm_initialization) {
    wasm_initialization = normalize_input(input).then(async (bytes) => {
      const { instance } = await WebAssembly.instantiate(bytes, create_wasi_shim());
      wasm_exports = instance.exports;
      return wasm_exports;
    });
  }

  return wasm_initialization;
}

export function shortest_path_wasm(node_count, from, to, weights, start, target) {
  if (!wasm_exports) {
    throw new Error("WASM-модуль ещё не инициализирован.");
  }

  if (from.length !== to.length || to.length !== weights.length) {
    throw new Error("Массивы рёбер должны иметь одинаковую длину.");
  }

  const transfer_started = performance.now();

  wasm_exports.reset_allocator();
  const from_pointer = copy_uint32_array(from);
  const to_pointer = copy_uint32_array(to);
  const weights_pointer = copy_float64_array(weights);
  const found_pointer = alloc_uint32();
  const distance_pointer = alloc_float64();
  const visited_count_pointer = alloc_uint32();

  const transfer_in_time = performance.now() - transfer_started;

  const compute_started = performance.now();
  const error_code = wasm_exports.run_shortest_path(
    node_count,
    from.length,
    from_pointer,
    to_pointer,
    weights_pointer,
    start,
    target,
    found_pointer,
```

```
    distance_pointer,
    visited_count_pointer
  );
  const compute_time = performance.now() - compute_started;

  if (error_code !== 0) {
    throw new Error(map_error_code(error_code));
  }

  const read_started = performance.now();
  const found = Boolean(new Uint32Array(get_memory().buffer, found_pointer, 1)[0]);
  const distance = new Float64Array(get_memory().buffer, distance_pointer, 1)[0];
  const visited_count = new Uint32Array(get_memory().buffer, visited_count_pointer,
1)[0];
  const read_out_time = performance.now() - read_started;

  console.info("READ STARTED: ", read_out_time)

  return {
    found,
    distance: found ? distance : null,
    visited_count,
    timings: {
      transfer: transfer_in_time + read_out_time,
      transfer_in: transfer_in_time,
      compute: compute_time,
      read_out: read_out_time
    }
  };
}
```

Вывод: в ходе работы были реализованы два решения задачи в браузере: на JavaScript и на WASM. Для сравнения производительности были сгенерированы большие наборы входных данных размером от 10 000 элементов. Тестирование показало, что обе реализации корректно выполняют поставленную задачу, но отличаются по скорости работы. JavaScript проще в разработке и отладке, а WASM может быть эффективнее при больших вычислительных нагрузках. Таким образом, выбор между JavaScript и WASM зависит от требований к производительности и удобству реализации.